

Lecture 2: Evolution of the Internet

Before the Internet existed, computers first needed a way to **represent information** and then a way to **communicate with each other**.

Storing Characters in Computers

Computers understand only **Binary (0s and 1s)**.

Numbers can easily be converted into binary, but what about characters like:

```
Hello
A
B
@
#
```

A standard was needed to represent these characters as binary values.

ASCII (American Standard Code for Information Interchange)

ASCII assigns a **unique decimal value** to every English character.

CHARACTER	ASCII
A	65
B	66
a	97
b	98
Space	32

Example:

HELLO

H → 72

E → 69

L → 76

L → 76

O → 79

These decimal values are converted into **8-bit binary** before being stored in memory.

ASCII can represent $2^8 = 256$ characters.

Limitation of ASCII

ASCII was designed mainly for English characters.

It cannot represent:

- ♦ Hindi (हिन्दी)
- ♦ Chinese (中文)
- ♦ Japanese (日本語)
- ♦ Emojis (😄)

Therefore, ASCII was **not sufficient** for a world where computers needed to handle every language and symbol, not just English.

Unicode & UTF

Unicode was introduced to represent characters from almost every language in the world.

Common formats:

- ♦ UTF-8
- ♦ UTF-16
- ♦ UTF-32

FORMAT	DESCRIPTION
UTF-8	Most commonly used on the Internet
UTF-16	Supports approximately 2^{16} code values
UTF-32	Supports 2^{32} code values

UTF stands for **Unicode Transformation Format**.

UTF-8 is preferred on the web mainly because it is backward-compatible with ASCII (the first **128 values** match exactly) while still being able to represent virtually any character.

Evolution of Programming Languages

1. Machine Language

- ✦ Written in Binary (0 and 1)
- ✦ Directly understood by computers
- ✦ Extremely difficult for humans to write or read

```
10101010
00101100
```

2. Assembly Language

Uses human-readable mnemonics instead of raw binary.

```
ADD 2, 3
```

Since computers understand only binary, an **Assembler** converts Assembly Language into Machine Language.

Just like a translator converts Hindi into English, an Assembler converts Assembly into Machine code.

3. Middle-Level Languages

- ✦ C and C++ lie between Assembly and High-Level Languages (HLL).
- ✦ They are not considered pure HLLs, since they still allow low-level operations like direct memory access.

These require a **Compiler** to convert code into Machine Language.

4. High-Level Languages

Examples:

- ✦ Java
- ✦ Python

These require:

- ✦ Compiler
- ✦ Interpreter

to convert the program into Machine Language.

Telephone Communication

Before the Internet, communication mainly happened through **Telephones**.



A dedicated physical communication line was required between sender and receiver.

Anyone connected to the same physical line could potentially listen to the conversation — **an early security concern**.

Telephone communication used **Analog Signals**.

Limitations:

- ♦ Expensive
- ♦ Dedicated path required
- ♦ Mainly voice communication

Cold War

During the Cold War:

- ♦ USA
- ♦ Soviet Union (USSR)

Both countries competed intensely in technology and space research.

The USA wanted a communication network that could continue working even if some parts were destroyed during a nuclear attack — **a network with no single point of failure**.

This strategic need led to the development of **ARPANET** in the 1960s.

ARPANET

ARPANET was created to connect universities and research centers.

Initially, it connected:

- ♦ UCLA
- ♦ Stanford Research Institute (SRI)
- ♦ University of California, Santa Barbara
- ♦ University of Utah

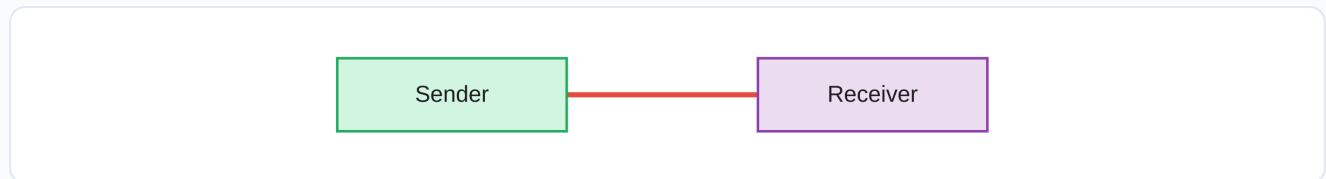
ARPANET became the foundation of today's Internet.

Circuit Switching

Telephone networks used **Circuit Switching**.

Working:

- ✦ A dedicated communication path is established between sender and receiver.
- ✦ The same path is used throughout the entire communication.



Signals:

- ✦ Telephone networks use **Analog Signals**.
- ✦ Computers understand **Digital Signals (Binary)**.

Therefore, a **Modem (MODulator-DEModulator)** is required to bridge the two.

Modem:

A Modem converts:

- ✦ Digital → Analog
- ✦ Analog → Digital

Advantages:

- ✦ Continuous communication
- ✦ Simple implementation

Disadvantages:

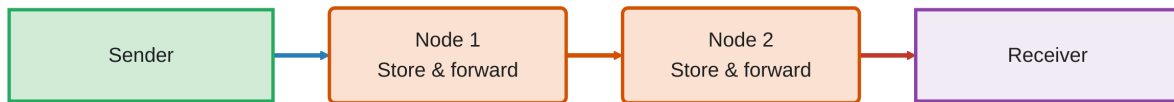
- ✦ Poor resource utilization (the path stays reserved even during silence)
- ✦ A single failure along the path breaks the entire communication
- ✦ Less secure, because the physical line can be tapped

Message Switching

Instead of creating a dedicated path:

- ✦ The entire message is stored at an intermediate node.
- ✦ It is then forwarded to the next node.

This technique is called **Store and Forward**.



Advantages:

- ♦ Better resource utilization compared to Circuit Switching
- ♦ No dedicated connection required

Disadvantages:

- ♦ High delay
- ♦ Large storage required at each node
- ♦ Receiver must wait until the complete message arrives

Example:

Like waiting for an entire video to download before you can watch it.

Packet Switching

ARPANET introduced **Packet Switching**.

Instead of sending one large message:

HELLO

The message is divided into **small packets**:

HELLO

↓

H E L L O

Each packet:

- ♦ Has its own header
- ♦ Can travel through a different route to the destination
- ♦ Is reassembled in the correct order at the destination

Advantages:

- ♦ Efficient bandwidth utilization
- ♦ Faster communication
- ♦ Reliable, since packets can be re-sent individually if lost
- ♦ Alternate routes are available if one path fails

- ◆ Overall better network performance

First Internet Message (1969)

The first message attempted to be sent over ARPANET was:

LOGIN

Only:

LO

was received before the system crashed.

Even though the message was incomplete, it is remembered as the first message ever sent over the Internet.

Circuit Switching vs Message Switching vs Packet Switching

CIRCUIT SWITCHING	MESSAGE SWITCHING	PACKET SWITCHING
Dedicated path	Store entire message	Divide into packets
Analog communication	Store & Forward	Multiple routes
Low delay after setup	High delay	Low delay
Telephone network	Early data transfer	Modern Internet

Summary

- ♦ ASCII stores English characters using 8-bit binary encoding but is limited to 256 characters.
- ♦ Unicode, through UTF formats (UTF-8, UTF-16, UTF-32), supports characters from almost every language.
- ♦ Programming languages evolved from Machine Language → Assembly Language → Middle-Level Languages → High-Level Languages.
- ♦ C and C++ lie between Assembly and High-Level Languages, and are not considered pure HLLs.
- ♦ Assemblers, Compilers, and Interpreters are all used to translate human-written programs into machine language.
- ♦ Telephone networks relied on Circuit Switching, which used a dedicated communication path and Analog signals.
- ♦ A Modem converts Digital signals to Analog signals and vice versa, enabling computers to use telephone lines.
- ♦ ARPANET, developed during the Cold War as a fault-tolerant communication network, became the foundation of the modern Internet.
- ♦ Circuit Switching guarantees a dedicated path but wastes network resources and is vulnerable to single points of failure.
- ♦ Message Switching uses the Store-and-Forward technique, improving resource use but introducing higher delay.
- ♦ Packet Switching divides data into small packets that can travel independently through different routes and be reassembled at the destination.
- ♦ Packet Switching is the backbone of the modern Internet, offering efficiency, reliability, and resilience.
- ♦ The first message ever sent over the Internet was "LO" (an incomplete "LOGIN"), transmitted over ARPANET in 1969.